Divisão e Conquista

PCC104 – Projeto e Análise de Algoritmos[1]

9 de abril de 2026

[1] Baseado em Cormen et al. *Algoritmos* (2022)

Sumário

Introdução

MergeSort

Recorrências

Indução Matemática

Método de Substituição

Método da Árvore de Recursão

Divisão e Conquista

Definição

Técnica de projeto de algoritmos que resolve problemas de forma **recursiva**, seguindo três etapas.

1. **Dividir:** o problema original em subproblemas menores (instâncias do mesmo problema).

Divisão e Conquista

Definição

Técnica de projeto de algoritmos que resolve problemas de forma **recursiva**, seguindo três etapas.

1. **Dividir:** o problema original em subproblemas menores (instâncias do mesmo problema).
2. **Conquistar:** resolver os subproblemas recursivamente. Quando suficientemente pequenos, resolver diretamente (**caso base**).

Divisão e Conquista

Definição

Técnica de projeto de algoritmos que resolve problemas de forma **recursiva**, seguindo três etapas.

1. **Dividir:** o problema original em subproblemas menores (instâncias do mesmo problema).
2. **Conquistar:** resolver os subproblemas recursivamente. Quando suficientemente pequenos, resolver diretamente (**caso base**).
3. **Combinar:** as soluções dos subproblemas para construir a solução do problema original.

MergeSort — Visão Geral

1. **Dividir:** arranjo de n elementos em dois subarranjos de $n/2$ cada.

MergeSort — Visão Geral

1. **Dividir:** arranjo de n elementos em dois subarranjos de $n/2$ cada.
2. **Conquistar:** ordenar cada metade recursivamente. Caso base: tamanho 1.

MergeSort — Visão Geral

1. **Dividir:** arranjo de n elementos em dois subarranjos de $n/2$ cada.
2. **Conquistar:** ordenar cada metade recursivamente. Caso base: tamanho 1.
3. **Combinar:** intercalar (*merge*) as duas metades ordenadas.

Etapa central

O trabalho real ocorre na **combinação** — o procedimento Merge.

Procedimento Merge(A, p, q, r)

Ideia

Intercalar $A[p \,..\, q]$ e $A[q{+}1 \,..\, r]$, ambos já ordenados, em $A[p \,..\, r]$.

1. Copiar as duas metades em arranjos auxiliares L e R

Procedimento Merge(A, p, q, r)

Ideia

Intercalar $A[p \,..\, q]$ e $A[q{+}1 \,..\, r]$, ambos já ordenados, em $A[p \,..\, r]$.

1. Copiar as duas metades em arranjos auxiliares L e R
2. Comparar elementos de L e R, copiando o menor de volta para A

Procedimento Merge(A, p, q, r)

Ideia

Intercalar $A[p\,..\,q]$ e $A[q{+}1\,..\,r]$, ambos já ordenados, em $A[p\,..\,r]$.

1. Copiar as duas metades em arranjos auxiliares L e R
2. Comparar elementos de L e R, copiando o menor de volta para A
3. Copiar elementos restantes de L ou R

Pseudocódigo — Merge(A, p, q, r)

$n_L = q - p + 1; \quad n_R = r - q$

Aloca $L[0\,..\,n_L{-}1]$, $R[0\,..\,n_R{-}1]$

for $i = 0$ **to** $n_L{-}1$

  $L[i] = A[p + i]$

for $j = 0$ **to** $n_R{-}1$

  $R[j] = A[q + j + 1]$

$i = 0;\ j = 0;\ k = p$

while $i < n_L$ **and** $j < n_R$

  if $L[i] \leq R[j]$: $A[k] = L[i];\ i{+}{+}$

  else: $A[k] = R[j];\ j{+}{+}$

  $k{+}{+}$

while $i < n_L$

  $A[k] = L[i];\ i{+}{+};\ k{+}{+}$

while $j < n_R$

  $A[k] = R[j];\ j{+}{+};\ k{+}{+}$

Pseudocódigo — MergeSort(A, p, r)

if $p \geq r$: **return** // caso base

$q = \lfloor (p + r)/2 \rfloor$

MergeSort(A, p, q) // metade esquerda

MergeSort$(A, q + 1, r)$ // metade direita

Merge(A, p, q, r) // combina

Análise de Complexidade do Merge

Resultado

O procedimento Merge executa em tempo $\Theta(n)$, onde $n = r - p + 1$.

Análise de Complexidade do Merge

Resultado

O procedimento Merge executa em tempo $\Theta(n)$, onde $n = r - p + 1$.

- Cada elemento é copiado para A **exatamente uma vez**
- Seja no laço principal, seja nos laços de finalização
- Total de cópias: $t_L + (n_L - t_L) + t_R + (n_R - t_R) = n_L + n_R = n$

Análise de Complexidade do Merge

Resultado

O procedimento Merge executa em tempo $\Theta(n)$, onde $n = r - p + 1$.

- Cada elemento é copiado para A **exatamente uma vez**
- Seja no laço principal, seja nos laços de finalização
- Total de cópias: $t_L + (n_L - t_L) + t_R + (n_R - t_R) = n_L + n_R = n$

Agrupando os custos: $T(n) = \alpha \cdot n + \beta = \Theta(n)$

Recorrência do MergeSort

A análise do MergeSort leva à recorrência:

$$T(n) = 2\,T(n/2) + \Theta(n), \quad T(1) = \Theta(1)$$

Recorrência do MergeSort

A análise do MergeSort leva à recorrência:

$$T(n) = 2\,T(n/2) + \Theta(n), \quad T(1) = \Theta(1)$$

Solução

Pelo Teorema Mestre (caso 2, $a = 2$, $b = 2$, $f(n) = \Theta(n)$):

$$T(n) = \Theta(n \lg n)$$

Como resolver recorrências como essa? $\longrightarrow$ Próximas seções.

Recorrências — Definição

Definição

Uma **recorrência** é uma equação que descreve $T(n)$ em termos de seu valor em entradas menores.

Recorrências — Definição

Definição

Uma **recorrência** é uma equação que descreve $T(n)$ em termos de seu valor em entradas menores.

Dois elementos essenciais:

- **Caso recursivo:** $T(n)$ em função de $T(m)$ com $m < n$

Recorrências — Definição

Definição

Uma **recorrência** é uma equação que descreve $T(n)$ em termos de seu valor em entradas menores.

Dois elementos essenciais:

- **Caso recursivo:** $T(n)$ em função de $T(m)$ com $m < n$
- **Caso base:** valor de $T(n)$ para entradas suficientemente pequenas

Recorrências — Exemplos

Busca linear

$$T(n) = T(n-1) + c$$

$$T(1) = c$$

$$\Rightarrow \; T(n) = \Theta(n)$$

Recorrências — Exemplos

Busca linear

$$T(n) = T(n-1) + c$$
$$T(1) = c$$
$$\Rightarrow \; T(n) = \Theta(n)$$

Merge-Sort

$$T(n) = 2\,T(n/2) + \Theta(n)$$
$$T(1) = \Theta(1)$$
$$\Rightarrow \; T(n) = \Theta(n \lg n)$$

Recorrências Bem e Mal Definidas

Bem definida

- Caso base definido
- Argumento sempre decresce em direção ao caso base
- Valor determinado de forma não ambígua

Recorrências Bem e Mal Definidas

Bem definida

- Caso base definido
- Argumento sempre decresce em direção ao caso base
- Valor determinado de forma não ambígua

Mal definida

- Sem caso base: $T(n) = 2T(n/2) + n$, sem condição de contorno
- Sem decrescimento: $T(n) = T(n) + 1$
- Domínio ambíguo: $T(n/2)$ com n ímpar $\Rightarrow$ usar $\lfloor \cdot \rfloor$, $\lceil \cdot \rceil$

Indução Matemática — Estrutura

Objetivo

Demonstrar que $P(n)$ vale para todo $n \geq n_0$.

Indução Matemática — Estrutura

Objetivo

Demonstrar que $P(n)$ vale para todo $n \geq n_0$.

1. **Caso base:** verificar que $P(n_0)$ é verdadeira.

Indução Matemática — Estrutura

Objetivo

Demonstrar que $P(n)$ vale para todo $n \geq n_0$.

1. **Caso base:** verificar que $P(n_0)$ é verdadeira.
2. **Passo indutivo:** assumir $P(k)$ (**hipótese indutiva**) e provar $P(k+1)$.

Indução Matemática — Estrutura

Objetivo

Demonstrar que $P(n)$ vale para todo $n \geq n_0$.

1. **Caso base:** verificar que $P(n_0)$ é verdadeira.
2. **Passo indutivo:** assumir $P(k)$ (**hipótese indutiva**) e provar $P(k+1)$.

Intuição: efeito dominó — o caso base derruba a primeira peça, o passo indutivo garante que cada peça derruba a seguinte.

Exemplo: Soma dos Primeiros n Naturais

Provar: $\displaystyle\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$ para todo $n \geq 1$.

Exemplo: Soma dos Primeiros n Naturais

Provar: $\displaystyle\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$ para todo $n \geq 1$.

Caso base ($n = 1$): $\sum_{i=1}^{1} i = 1 = \frac{1 \cdot 2}{2}$. ✓

Exemplo: Soma dos Primeiros n Naturais

Provar: $\displaystyle\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$ para todo $n \geq 1$.

Caso base ($n = 1$): $\sum_{i=1}^{1} i = 1 = \frac{1 \cdot 2}{2}$. ✓

Passo indutivo: assumindo que vale para k:

$$\sum_{i=1}^{k+1} i = \underbrace{\sum_{i=1}^{k} i}_{=k(k+1)/2} + (k+1) = \frac{k(k+1) + 2(k+1)}{2} = \frac{(k+1)(k+2)}{2} \quad \checkmark$$

Indução Forte

Diferença

Na **indução forte**, a hipótese assume que $P(m)$ vale para **todo** m com $n_0 \leq m \leq k$ (não apenas para k).

Indução Forte

Diferença

Na **indução forte**, a hipótese assume que $P(m)$ vale para **todo** m com $n_0 \leq m \leq k$ (não apenas para k).

- Logicamente equivalente à indução simples

Indução Forte

Diferença

Na **indução forte**, a hipótese assume que $P(m)$ vale para **todo** m com $n_0 \leq m \leq k$ (não apenas para k).

- Logicamente equivalente à indução simples
- Essencial para recorrências: $T(n)$ pode depender de $T(\lfloor n/2 \rfloor)$ e $T(\lceil n/2 \rceil)$ simultaneamente

Indução Forte

Diferença

Na **indução forte**, a hipótese assume que $P(m)$ vale para **todo** m com $n_0 \leq m \leq k$ (não apenas para k).

- Logicamente equivalente à indução simples
- Essencial para recorrências: $T(n)$ pode depender de $T(\lfloor n/2 \rfloor)$ e $T(\lceil n/2 \rceil)$ simultaneamente
- Precisamos que a hipótese valha para **todas** as entradas menores

Exemplo com Indução Forte

Proposição

Todo inteiro $n \geq 2$ pode ser escrito como produto de primos.

Exemplo com Indução Forte

Proposição

Todo inteiro $n \geq 2$ pode ser escrito como produto de primos.

Caso base ($n = 2$): 2 é primo. ✓

Exemplo com Indução Forte

Proposição

Todo inteiro $n \geq 2$ pode ser escrito como produto de primos.

Caso base ($n = 2$): 2 é primo. ✓

Passo indutivo: suponha que vale para todo m com $2 \leq m \leq k$.

- Se $k + 1$ é primo $\Rightarrow$ imediato.

Exemplo com Indução Forte

Proposição

Todo inteiro $n \geq 2$ pode ser escrito como produto de primos.

Caso base ($n = 2$): 2 é primo. ✓

Passo indutivo: suponha que vale para todo m com $2 \leq m \leq k$.

- Se $k + 1$ é primo $\Rightarrow$ imediato.
- Se não: $k + 1 = a \cdot b$ com $2 \leq a, b \leq k$.
- Pela hipótese, a e b são produtos de primos $\Rightarrow$ $k + 1$ também. ✓

Exemplo com Indução Forte

Proposição

Todo inteiro $n \geq 2$ pode ser escrito como produto de primos.

Caso base ($n = 2$): 2 é primo. ✓

Passo indutivo: suponha que vale para todo m com $2 \leq m \leq k$.

- Se $k + 1$ é primo $\Rightarrow$ imediato.
- Se não: $k + 1 = a \cdot b$ com $2 \leq a, b \leq k$.
- Pela hipótese, a e b são produtos de primos $\Rightarrow$ $k + 1$ também. ✓

Nota: indução simples não bastaria — saber que k é produto de primos nada diz sobre $k + 1$.

Indução e Recorrências

Provar que $T(n) \leq c\,g(n)$ para uma recorrência é uma **prova por indução forte**:

1. Assumir $T(m) \leq c\,g(m)$ para todo $m < n$

Indução e Recorrências

Provar que $T(n) \leq c\,g(n)$ para uma recorrência é uma **prova por indução forte**:

1. Assumir $T(m) \leq c\,g(m)$ para todo $m < n$
2. Substituir na recorrência

Indução e Recorrências

Provar que $T(n) \leq c\,g(n)$ para uma recorrência é uma **prova por indução forte**:

1. Assumir $T(m) \leq c\,g(m)$ para todo $m < n$
2. Substituir na recorrência
3. Verificar que $T(n) \leq c\,g(n)$ segue algebricamente

Indução e Recorrências

Provar que $T(n) \leq c\,g(n)$ para uma recorrência é uma **prova por indução forte**:

1. Assumir $T(m) \leq c\,g(m)$ para todo $m < n$
2. Substituir na recorrência
3. Verificar que $T(n) \leq c\,g(n)$ segue algebricamente
4. Verificar o caso base, ajustando c se necessário

$\longrightarrow$ Esta é a mecânica do **método de substituição**.

Método de Substituição — Estrutura

Dois passos

1. **Conjecturar** a forma da solução: $T(n) \leq c \cdot g(n)$

Método de Substituição — Estrutura

Dois passos

1. **Conjecturar** a forma da solução: $T(n) \leq c \cdot g(n)$
2. **Provar por indução** que a conjectura está correta, determinando as constantes

Exemplo: Recorrência do Merge-Sort

$$T(n) = 2\,T(\lfloor n/2 \rfloor) + dn, \quad T(1) = a$$

Conjectura: $T(n) \leq c\, n \lg n$ para todo $n \geq 2$.

Exemplo: Recorrência do Merge-Sort

$$T(n) = 2\,T(\lfloor n/2 \rfloor) + dn, \quad T(1) = a$$

Conjectura: $T(n) \leq c\,n \lg n$ para todo $n \geq 2$.

Passo indutivo: assumindo $T(\lfloor n/2 \rfloor) \leq c\,\lfloor n/2 \rfloor \lg \lfloor n/2 \rfloor$:

$$\begin{aligned}
T(n) &\leq 2\big(c\,\lfloor n/2 \rfloor \lg \lfloor n/2 \rfloor\big) + dn \\
&\leq c\,n \lg(n/2) + dn \\
&= c\,n \lg n - c\,n + dn \\
&= c\,n \lg n - (c - d)\,n \\
&\leq c\,n \lg n \quad \checkmark \qquad (\text{desde que } c \geq d)
\end{aligned}$$

Exemplo: Caso Base

Para $n = 1$: $T(1) \leq c \cdot 1 \cdot \lg 1 = 0$ — falha! (pois $T(1) = a > 0$)

Exemplo: Caso Base

Para $n = 1$: $T(1) \leq c \cdot 1 \cdot \lg 1 = 0$ — falha! (pois $T(1) = a > 0$)

Para $n = 2$: $T(2) = 2a + 2d \leq c \cdot 2 \lg 2 = 2c$
$\Rightarrow$ vale para $c \geq a + d$

Exemplo: Caso Base

Para $n = 1$: $T(1) \leq c \cdot 1 \cdot \lg 1 = 0$ — **falha!** (pois $T(1) = a > 0$)

Para $n = 2$: $T(2) = 2a + 2d \leq c \cdot 2 \lg 2 = 2c$
$\Rightarrow$ vale para $c \geq a + d$

Conclusão

Com $n_0 = 2$ e $c \geq a + d \geq d$: $\quad T(n) \leq c\, n \lg n$ para todo $n \geq 2$.
Logo $T(n) = O(n \lg n)$.

Flexibilidade no Caso Base

Por que a falha em $n = 1$ não é problema?

A notação $T(n) = O(g(n))$ exige apenas que $T(n) \leq c\, g(n)$ para $n \geq n_0$.

Flexibilidade no Caso Base

Por que a falha em $n = 1$ não é problema?

A notação $T(n) = O(g(n))$ exige apenas que $T(n) \leq c\, g(n)$ para $n \geq n_0$.

- Podemos escolher n_0 e c

Flexibilidade no Caso Base

Por que a falha em $n = 1$ não é problema?

A notação $T(n) = O(g(n))$ exige apenas que $T(n) \leq c\, g(n)$ para $n \geq n_0$.

- Podemos **escolher** n_0 e c
- Um número finito de condições de contorno sempre pode ser acomodado tomando c grande o suficiente

Flexibilidade no Caso Base

Por que a falha em $n = 1$ não é problema?

A notação $T(n) = O(g(n))$ exige apenas que $T(n) \leq c\, g(n)$ para $n \geq n_0$.

- Podemos **escolher** n_0 e c
- Um número finito de condições de contorno sempre pode ser acomodado tomando c grande o suficiente
- Na prática: concentrar-se no **passo indutivo** e tratar o caso base como verificação técnica

Armadilhas Comuns

Hipótese que não fecha

Provar $T(n) = 2T(\lfloor n/2 \rfloor) + n$ é $O(n)$:
$T(n) \leq c\,n + n = (c+1)\,n$
Não prova $T(n) \leq c\,n$ para a mesma constante!

Armadilhas Comuns

Hipótese que não fecha

Provar $T(n) = 2T(\lfloor n/2 \rfloor) + n$ é $O(n)$:
$T(n) \leq c\,n + n = (c+1)\,n$
Não prova $T(n) \leq c\,n$ para a mesma constante!

Subtrair termo de ordem inferior

Para
$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$:
Conjecturar $T(n) \leq c\,n$ não fecha.
Mas $T(n) \leq c\,n - d$ fornece a **margem** necessária.

Como Obter Boas Conjecturas

1. **Similaridade:** recorrência parecida com uma conhecida? Use a mesma forma.
   Ex.: $T(n) = 2T(\lfloor n/2 \rfloor + 17) + n \Rightarrow$ provavelmente $O(n \lg n)$

Como Obter Boas Conjecturas

1. **Similaridade:** recorrência parecida com uma conhecida? Use a mesma forma.
   Ex.: $T(n) = 2T(\lfloor n/2 \rfloor + 17) + n \Rightarrow$ provavelmente $O(n \lg n)$
2. **Limitantes frouxos:** provar $O(n^2)$ e $\Omega(n)$ primeiro, depois refinar.

Como Obter Boas Conjecturas

1. **Similaridade:** recorrência parecida com uma conhecida? Use a mesma forma.
   Ex.: $T(n) = 2T(\lfloor n/2 \rfloor + 17) + n \Rightarrow$ provavelmente $O(n \lg n)$
2. **Limitantes frouxos:** provar $O(n^2)$ e $\Omega(n)$ primeiro, depois refinar.
3. **Árvore de recursão:** gerar a conjectura visualmente, depois verificar por substituição.

Árvore de Recursão — Ideia

Definição

Expandir a recorrência em uma **árvore** cujos nós representam os custos em cada nível de recursão.

Árvore de Recursão — Ideia

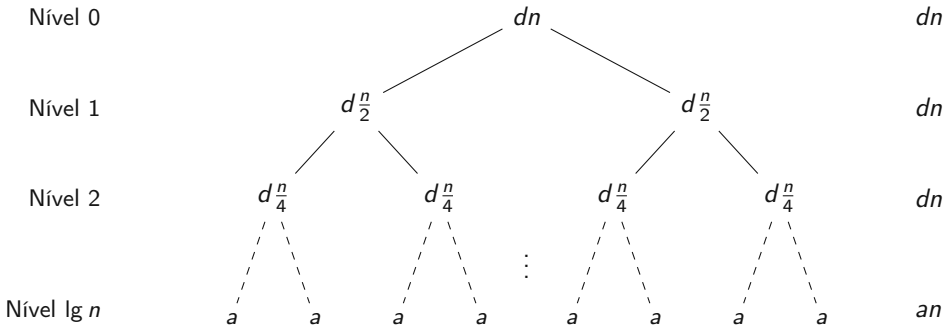
Definição

Expandir a recorrência em uma **árvore** cujos nós representam os custos em cada nível de recursão.

Para $T(n) = a\,T(n/b) + f(n)$:

- **Raiz:** custo $f(n)$
- Cada nó gera a filhos (subproblemas de tamanho m/b)
- **Folhas:** caso base, custo $\Theta(1)$ cada
- Custo total = soma dos custos de todos os níveis

Exemplo: $T(n) = 2\,T(n/2) + dn$



Exemplo: $T(n) = 2\,T(n/2) + dn$ — Análise

- Nível i: 2^i nós, cada um com custo $d(n/2^i)$
- Custo do nível i: $2^i \cdot d(n/2^i) = dn$ (constante!)

Exemplo: $T(n) = 2\,T(n/2) + dn$ — Análise

- Nível i: 2^i nós, cada um com custo $d(n/2^i)$
- Custo do nível i: $2^i \cdot d(n/2^i) = dn$ (constante!)
- Profundidade: $n/2^i = 1 \Rightarrow i = \lg n$
- Folhas: $2^{\lg n} = n$ folhas, custo total an

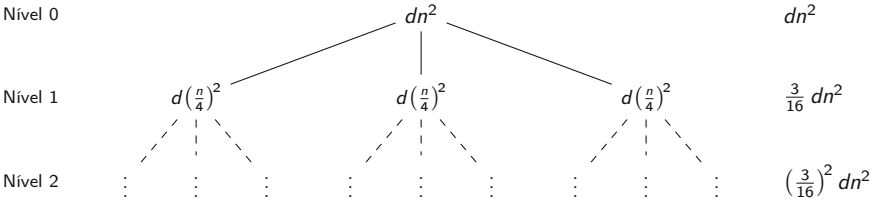
Exemplo: $T(n) = 2\,T(n/2) + dn$ — Análise

- Nível i: 2^i nós, cada um com custo $d(n/2^i)$
- Custo do nível i: $2^i \cdot d(n/2^i) = dn$ (constante!)
- Profundidade: $n/2^i = 1 \Rightarrow i = \lg n$
- Folhas: $2^{\lg n} = n$ folhas, custo total an

Custo total

$$T(n) = \underbrace{dn + dn + \cdots + dn}_{\lg n \text{ parcelas}} + an = dn \lg n + an = \Theta(n \lg n)$$

Exemplo: $T(n) = 3\,T(n/4) + dn^2$



Exemplo: $T(n) = 3\,T(n/4) + dn^2$ — Análise

- Nível i: 3^i nós, custo por nó $d(n/4^i)^2 \Rightarrow$ total $\left(\frac{3}{16}\right)^i dn^2$

Exemplo: $T(n) = 3\,T(n/4) + dn^2$ — Análise

- Nível i: 3^i nós, custo por nó $d(n/4^i)^2 \Rightarrow$ total $\left(\frac{3}{16}\right)^i dn^2$
- Profundidade: $\log_4 n$
- Folhas: $3^{\log_4 n} = n^{\log_4 3}$

Exemplo: $T(n) = 3\,T(n/4) + dn^2$ — Análise

- Nível i: 3^i nós, custo por nó $d(n/4^i)^2 \Rightarrow$ total $\left(\frac{3}{16}\right)^i dn^2$
- Profundidade: $\log_4 n$
- Folhas: $3^{\log_4 n} = n^{\log_4 3}$

Série geométrica com razão $3/16 < 1$:

$$\sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i = \frac{16}{13}$$

Exemplo: $T(n) = 3\,T(n/4) + dn^2$ — Análise

- Nível i: 3^i nós, custo por nó $d(n/4^i)^2 \Rightarrow$ total $\left(\frac{3}{16}\right)^i dn^2$
- Profundidade: $\log_4 n$
- Folhas: $3^{\log_4 n} = n^{\log_4 3}$

Série geométrica com razão $3/16 < 1$:

$$\sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i = \frac{16}{13}$$

Resultado

$T(n) \leq \frac{16}{13}\,dn^2 + \Theta(n^{\log_4 3}) = O(n^2)$

Custo **dominado pela raiz** — caso típico do caso 3 do Teorema Mestre.

Árvore de Recursão como Conjectura

Uso heurístico

A árvore fornece uma **conjectura** para a solução, que deve ser verificada pelo método de substituição.

Uso como prova

Com cuidado (pisos, tetos, casos base), a própria árvore constitui uma **prova válida**.

Próxima seção: Teorema Mestre — formaliza os três cenários.

Referências I

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Algoritmos: teoria e prática*. LTC, 4 edition.